

深圳大学实验报告

课程名称: Java 程序设计

实验项目名称: 课程实验 4: I/O、GUI 和网络编程

学院: 数学科学学院

专业: 信息与计算科学 (数学与计算机实验班)

指导教师: 潘微科

报告人: 黄嘉聪 学号: 2023192044 班级: 1

实验时间: 2024 年 11 月 29 日 (周五) - 2024 年 12 月 18 日 (周三)

实验报告提交时间: 2024 年 12 月 18 日

教务部制

实验目的与要求:

实验目的: 掌握 I/O 程序设计, 能够读写本地文件等, 初步掌握图形界面程序设计, 掌握网络通信协议及相关程序设计。

实验要求:

Part 1 (25 分)

(1.1). 数据解析和统计。 <https://snap.stanford.edu/data/web-Amazon.html> 网站上有很多 Amazon 的数据集供研究人员下载使用。本次实验使用 Watches.txt.gz 数据集, 请下载后解压。格式说明请看网页上的 “Data Format” 部分。在报告中附上 **程序截图**、运行结果 (如 **每个输出文件前 10 行的截图**等) 和 **简要文字说明**。

(i) 使用 Java 语言读取解压后的文件 (Watches.txt), 并得到以下文件 (10 分):

review.txt: 每行 2 列, 以分号作为分隔符, 第 1 列是 userID, 第 2 列是 productID, 表示(user, product)二元组。该文件中不同行之间的顺序, 按照 userID 从小到大排列, 当 userID 相同时按照 productID 从小到大排列。

注: 删除 ID 为 unknown 的记录; 排序时按照字符串顺序。

(ii) 使用 Java 语言根据 review.txt 进行计算, 并得到以下文件 (15 分):

productNeighborhood.txt: 每行 4 列, 以分号作为分隔符, 第 1 列是 productID, 第 2-4 列是与该 product 最相似 (根据相似度值) 的 3 个 product 的 productID, 按相似度值从大到小排列 (当相似度相同时, 按照 productID 的字符串顺序从小到大排序), 其中相似度是通过 review.txt 中的两列的信息计算得到的 Jaccard index 值。该文件中不同行之间的顺序, 按照第 1 列的 productID 从小到大排列。

关于相似度的计算, 要求使用多线程实现 (5 分)。

注: 计算两个商品 (product) 之间的相似度 (即 Jaccard index) 时, 根据这两个商品所关联的用户的集合来计算——集合的交集的大小除以集合的并集的大小。

Part 2 (25 分)

(2.1).使用 JLabel、JTextArea、JButton 等控件实现句子中英互译的 demo, 该 demo 包含两个文本框, 第一个文本框用于输入中文句子或显示第二个文本框中的英文句子的中文翻译, 第二个文本框用于输入英文句子或显示第一个文本框中的中文句子的英文翻译。每个文本框下方各有一个按钮, 第一个按钮的名称是 “中译英”, 第二个按钮的名称是 “英译中”, 点击按钮表示将该文本框中的内容翻译成另一种语言。要求使用以下三种方式:

A、使用自己事先准备好的中英文翻译 (此部分占 5 分);

B、使用百度翻译 API、有道翻译 API 或其他 API 中的一个 API (此部分占 5 分);

C、使用腾讯混元大模型、百度文心一言大模型 API、阿里通义千问大模型 API 或其他大模型 API 中的两个 API (此部分占 10 分);

要求使用以下两个例句:

建校 41 年, 深圳大学秉承 “自立、自律、自强” 的校训, 紧随特区, 锐意改革、快速发展, 为特区发展和国家现代化建设做出了重要贡献。

Sticking to the motto of “self-reliance, self-discipline, self-improvement”, the University is dedicated to serving the Shenzhen Special Economic Zone (SEZ), demonstrating China’s reform and opening up and pioneering change in higher education.

要求使用图形用户界面, 界面美观、交互友好。在报告中附上程序截图、运行结果和详细的文字说明。(5 分)

Part 3 (30 分)

(3.1). 利用套接字连接(TCP)编写程序,该程序包括三个客户端(ClientA、ClientB、ClientC)和一个服务端(ServerS),三个客户端通过服务端作为桥梁实现相互间的文字交流,例如,ClientA 先发信息给 ServerS,然后 ServerC 再将收到的信息转发给 ClientB 和 ClientC。在报告中附上示意图(三个客户端+一个服务端)、程序截图、完整的运行结果和简要文字说明。(20 分)

(3.2). 利用数据报通信(UDP)实现题(1)中的要求。(10 分)

报告写作。要求: 主要思路有明确的说明,重点代码有详细的注释,行文逻辑清晰可读性强,报告整体写作较为专业。(20 分)

说明:

- (1) 本次实验课作业满分为 100 分, 占总成绩的比例 7%。
- (2) 本次实验课作业截至时间 2024 年 12 月 18 日(周三) 21:59。
- (3) 报告正文: 请在**指定位置填写**, 本次实验**不需要单独提交源程序文件**。
- (4) 个人信息: WORD 文件名中的“姓名”、“学号”, 请改为你的**姓名和学号**; 实验报告的首页, **请准确填写“学院”、“专业”、“报告人”、“学号”、“班级”、“实验报告提交时间”**等信息。
- (5) 提交方式: 截至时间前, 请在 Blackboard 平台中提交。
- (6) 发现抄袭(包括复制&粘贴整句话、整张图), **抄袭者和被抄袭者的成绩记零分**。
- (7) 延迟提交, 不得分; 如有特殊情况, 请于截止日期之后的 **48 小时内**发邮件到 panweike@szu.edu.cn, 并在邮件中注明课程名称、作业名称、姓名、学号等信息, 以及特殊情况的说明, 我收到后会及时回复。
- (8) 期末考试阶段补交无效。

Part 1 (25 分)

(1.1). 数据解析和统计。 <https://snap.stanford.edu/data/web-Amazon.html> 网站上有很多 Amazon 的数据集供研究人员下载使用。本次实验使用 Watches.txt.gz 数据集，请下载后解压。格式说明请看网页上的“Data Format”部分。在报告中附上程序截图、运行结果（如每个输出文件前 10 行的截图等）和简要文字说明。

(i) 使用 Java 语言读取解压后的文件（Watches.txt），并得到以下文件（10 分）：

review.txt: 每行 2 列，以分号作为分隔符，第 1 列是 userID，第 2 列是 productID，表示(user, product)二元组。该文件中不同行之间的顺序，按照 userID 从小到大排列，当 userID 相同时按照 productID 从小到大排列。

注：删除 ID 为 unknown 的记录；排序时按照字符串顺序。

程序说明：

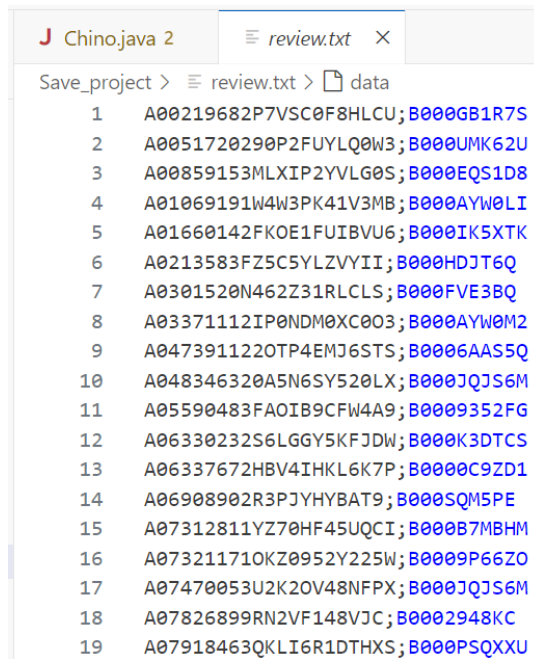
程序首先使用 Files 对于 Watches.txt 进行读取，然后使用列表进行存储，接着按照要求 userID、productID 进行读取后排序，再按顺序写入到 BufferedWriter 并保存

程序内容：

```
1 package com.chino;
2
3 import java.io.*;
4 import java.nio.file.*;
5 import java.util.*;
6
7 public class Chino {
8     Run | Debug | 运行 main | 调试 main,
9     public static void main(String[] args) throws IOException {
10         // 读取Watches.txt文件的所有行
11         List<String> lines = Files.readAllLines(Paths.get(first:"D:\\JAVA\\src\\com\\chino\\Watches.txt"));
12
13         // 用于存储userID和productID的列表
14         List<String[]> userProductPairs = new ArrayList<>();
15
16         // 临时变量，用于存储当前的productID和userID
17         String currentProductId = null;
18         String currentUserId = null;
19
20         // 遍历每一行，提取productID和userID
21         // 遍历每一行
22         for (String line : lines) {
23             // 如果行以"product/productId: "开头
24             if (line.startsWith(prefix:"product/productId: ")) {
25                 // 获取productID
26                 currentProductId = line.substring(beginIndex:18).trim();
27             } else if (line.startsWith(prefix:"review/userId: ")) {
28                 // 获取userID
29                 currentUserId = line.substring(beginIndex:15).trim();
30                 // 如果userID不是unknown且productID不为空
31                 if (!"unknown".equals(currentUserId) && currentProductId != null) {
32                     // 将userID和productID添加到userProductPairs中
33                     userProductPairs.add(new String[]{currentUserId, currentProductId});
34                 }
35             }
36         }
37
38         // 按照userID和productID排序
39         userProductPairs.sort(Comparator.comparing((String[] pair) -> pair[0])
40             .thenComparing(pair -> pair[1]));
41
42         // 创建一个BufferedWriter对象，用于写入review.txt文件
43         try (BufferedWriter writer = Files.newBufferedWriter(Paths.get(first:"D:\\JAVA\\src\\com\\chino\\review.txt"))) {
44             // 遍历过滤后的行，将每行的userID和productID写入review.txt文件
45             for (String[] pair : userProductPairs) {
46                 writer.write(pair[0] + ";" + pair[1]);
47                 writer.newLine();
48             }
49         }
50     }
51 }
```

图一 1.1.1 程序内容

输出结果:



```
Save_project > review.txt > data
1 A00219682P7VSC0F8HLCU;B000GB1R7S
2 A0051720290P2FUYLQ0W3;B000UMK62U
3 A00859153MLXIP2YVLG0S;B000EQS1D8
4 A01069191W4W3PK41V3MB;B000AYW0LI
5 A01660142FKOE1FUIBVU6;B000IK5XTK
6 A0213583FZ5C5YLZVYII;B000HDJT6Q
7 A0301520N462Z31RLCLS;B000FVE3BQ
8 A03371112IP0NDM0XC003;B000AYW0M2
9 A0473911220TP4EMJ6STS;B0006AAS5Q
10 A048346320A5N6SY520LX;B000JQJS6M
11 A05590483FAOIB9CFW4A9;B0009352FG
12 A06330232S6LGGY5KFJDW;B000K3DTCs
13 A06337672HBV4IHKL6K7P;B0000C9ZD1
14 A06908902R3PJYHYBAT9;B000SQM5PE
15 A07312811YZ70HF45UQCI;B000B7MBHM
16 A073211710KZ0952Y225W;B0009P66ZO
17 A07470053U2K20V48NFPX;B000JQJS6M
18 A07826899RN2VF148VJC;B0002948KC
19 A07918463QKLI6R1DTHXS;B000PSQXXU
```

图二 1.1.1 程序输出 (review.txt)

(ii) 使用 Java 语言根据 review.txt 进行计算, 并得到以下文件 (15 分):

productNeighborhood.txt: 每行 4 列, 以分号作为分隔符, 第 1 列是 productID, 第 2-4 列是与该 product 最相似 (根据相似度值) 的 3 个 product 的 productID, 按相似度值从大到小排列 (当相似度相同时, 按照 productID 的字符串顺序从小到大排序), 其中相似度是通过 review.txt 中的两列的信息计算得到的 Jaccard index 值。该文件中不同行之间的顺序, 按照第 1 列的 productID 从小到大排列。关于相似度的计算, 要求使用多线程实现 (5 分)。

注: 计算两个商品 (product) 之间的相似度 (即 Jaccard index) 时, 根据这两个商品所关联的用户的集合来计算——集合的交集的大小除以集合的并集的大小。

程序说明:

程序首先使用 Files 读取 review.txt 文件的所有行, 然后使用列表进行存储。接着, 程序按照 userID 和 productID 进行读取, 并构建用户-商品映射和商品-用户映射。然后, 程序计算每个商品与其他商品的相似度, 并找出最相似的三个商品。最后, 程序将每个商品最相似的三个商品按顺序写入到 BufferedWriter 并保存到 productNeighborhood.txt 文件中。

程序内容:

```
1 package com.chino;
2
3 import java.io.*;
4 import java.nio.file.*;
5 import java.util.*;
6 import java.util.concurrent.*;
7 import java.util.stream.*;
8
9 public class Chino_1 {
10     Run [Debug] 运行 main | 测试 main
11     public static void main(String[] args) throws IOException, InterruptedException {
12         // 读取review.txt文件的所有行
13         List<String> lines = Files.readAllLines(Paths.get(first:"D:\\JAVA\\src\\com\\chino\\review.txt"));
14
15         // 用于存储用户-商品映射和商品-用户映射
16         Map<String, Set<String>> userProducts = new HashMap<>();
17         Map<String, Set<String>> productUsers = new HashMap<>();
18
19         // 遍历每一行，构建用户-商品映射和商品-用户映射
20         for (String line : lines) {
21             String[] parts = line.split(regex:";");
22             String user = parts[0];
23             String product = parts[1];
24
25             userProducts.computeIfAbsent(user, k -> new HashSet<>()).add(product);
26             productUsers.computeIfAbsent(product, k -> new HashSet<>()).add(user);
27         }
28
29         // 获取所有商品的列表
30         List<String> products = new ArrayList<>(productUsers.keySet());
31         // 用于存储每个商品的最相似的三个商品
32         Map<String, List<String>> productNeighbors = new ConcurrentHashMap<>();
33
34         // 创建一个线程池
35         ExecutorService executor = Executors.newFixedThreadPool(Runtime.getRuntime().availableProcessors());
36         List<Future<?>> futures = new ArrayList<>();
37
38         // 遍历每个商品，计算其与其他商品的相似度
39         for (String product : products) {
40             futures.add(executor.submit(() -> {
41                 List<String> neighbors = products.stream()
42                     .filter(other -> !other.equals(product))
43                     .map(other -> new AbstractMap.SimpleEntry<>(other, jaccardIndex(productUsers.get(product), productUsers.get(other))))
44                     .sorted(Comparator.comparing((AbstractMap.SimpleEntry<String, Double> entry) -> -entry.getValue()))
45                     .limit(maxSize:3)
46                     .map(Map.Entry::getKey)
47                     .collect(Collectors.toList());
48                 productNeighbors.put(product, neighbors);
49             }));
50         }
51
52         // 等待所有任务完成，并处理可能的异常
53         for (Future<?> future : futures) {
54             try {
55                 future.get();
56             } catch (ExecutionException e) {
57                 e.printStackTrace();
58             }
59         }
60
61         // 关闭线程池
62         executor.shutdown();
63
64         // 创建一个BufferedWriter对象，用于写入productNeighborhood.txt文件
65         try (BufferedWriter writer = Files.newBufferedWriter(Paths.get(first:"D:\\JAVA\\src\\com\\chino\\productNeighborhood.txt"))) {
66             // 按照productId排序，并将每个商品的最相似的三个商品写入productNeighborhood.txt文件
67             for (String product : products.stream().sorted().collect(Collectors.toList())) {
68                 List<String> neighbors = productNeighbors.get(product);
69                 writer.write(product + ";" + String.join(delimiter:";", neighbors));
70                 writer.newLine();
71             }
72         }
73
74         // 计算两个集合之间的Jaccard相似度
75         private static double jaccardIndex(Set<String> set1, Set<String> set2) {
76             Set<String> intersection = new HashSet<>(set1);
77             intersection.retainAll(set2);
78             Set<String> union = new HashSet<>(set1);
79             union.addAll(set2);
80             return (double) intersection.size() / union.size();
81         }
82     }
83 }
```

图三 1.1.2 程序内容

输出结果:

```
J Chino.java 2 J Chino_1.java 2 productNeighborhood.txt X
Save_project > productNeighborhood.txt > data
1 0789208628;B000GXDYSQ;B000ERGGIY;B000H28UNU
2 1861262337;B000OVPGGI;0789208628;4971850491
3 4971850491;B000BM7KAU;B000EQVZOU;B000BQVU8Y
4 4971850788;B0007RTC50;0789208628;1861262337
5 4971850813;0789208628;1861262337;4971850491
6 7966770882;B000JZ7PEA;B000EQRZVC;B000E4Z3YA
7 7966780883;B000EQVZ86;B0007IR4U4;B000A7G4SI
8 B000026D6I;0789208628;1861262337;4971850491
9 B00004TF4K;0789208628;1861262337;4971850491
10 B00004WFX2;0789208628;1861262337;4971850491
11 B000051SEP;B0000YXU5S;B0000YXU8A;B0000YXU8U
12 B000051SES;B000EJNL8U;B000051SEP;0789208628
13 B000053ZFX;0789208628;1861262337;4971850491
14 B00005N6ZK;0789208628;1861262337;4971850491
15 B00005N6ZH;0789208628;1861262337;4971850491
16 B00005N9E2;B000GDC5LS;B000GDA17I;0789208628
17 B000050T8S;0789208628;1861262337;4971850491
18 B000050T8V;0789208628;1861262337;4971850491
19 B000050T8W;0789208628;1861262337;4971850491
```

图四 1.1.2 程序输出 (productNeighborhood.txt)

Part 2 (25 分)

(2.1).使用 JLabel、JTextArea、JButton 等控件实现句子中英互译的 demo，该 demo 包含两个文本框，第一个文本框用于输入中文句子或显示第二个文本框中的英文句子的中文翻译，第二个文本框用于输入英文句子或显示第一个文本框中的中文句子的英文翻译。每个文本框下方各有一个按钮，第一个按钮的名称是“中译英”，第二个按钮的名称是“英译中”，点击按钮表示将该文本框中的内容翻译成另一种语言。要求使用以下三种方式：

A、使用自己事先准备好的中英文翻译（此部分占 5 分）；

B、使用百度翻译 API、有道翻译 API 或其他 API 中的一个 API（此部分占 5 分）；

C、使用腾讯混元大模型、百度文心一言大模型 API、阿里通义千问大模型 API 或其他大模型 API 中的两个 API（此部分占 10 分）；

要求使用以下两个例句：

建校 41 年，深圳大学秉承“自立、自律、自强”的校训，紧随特区，锐意改革、快速发展，为特区发展和国家现代化建设做出了重要贡献。

Sticking to the motto of “self-reliance, self-discipline, self-improvement”, the University is dedicated to serving the Shenzhen Special Economic Zone (SEZ), demonstrating China’s reform and opening up and pioneering change in higher education.

要求使用图形用户界面，界面美观、交互友好。在报告中附上程序截图、运行结果和详细的文字说明。（5 分）

A、使用自己事先准备好的中英文翻译（此部分占 5 分）；

程序说明：

程序首先创建一个包含所给例句翻译对的静态映射 translations，然后在 main 方法中创建一个 JFrame 对象，并设置其标题、关闭操作、大小和布局。接着，程序创建多个 Swing 组件，包括标签、文本区域和按钮，并将它们添加到 JFrame 中。按钮的点击事件会触发翻译操作，获取文本区域中的内容，并从 translations 映射中查找对应的翻译结果，将结果显示在相应的文本区域中。最后，设置窗口可见。

程序内容：

```
1 package com.chino;
2
3 import java.awt.GridLayout;
4 import java.awt.event.ActionEvent;
5 import java.awt.event.ActionListener;
6 import java.util.HashMap;
7 import java.util.Map;
8
9 import javax.swing.JButton;
10 import javax.swing.JFrame;
11 import javax.swing.JLabel;
12 import javax.swing.JTextArea;
13
14 public class Chino {
15     // 定义一个Map, 用于存储中文和英文的翻译
16     private static final Map<String, String> translations = new HashMap<>();
17
18     // 静态代码块, 初始化Map
19     static {
20         // 将中文翻译存储到Map中
21         translations.put(key:"建校41年,深圳大学秉承“自立、自律、自强”的校训,紧随特区,锐意改革、快速发展,为特区发展",
22             value:"Sticking to the motto of “self-reliance, self-discipline, self-improvement”, the University is dedicated to serving the Shenzhen Special Economic Zone (SEZ), demonstrating China’s reform and opening up and pioneering change in higher education.");
23         translations.put(key:"Sticking to the motto of “self-reliance, self-discipline, self-improvement”, the University is dedicated to serving the Shenzhen Special Economic Zone (SEZ), demonstrating China’s reform and opening up and pioneering change in higher education.",
24             value:"建校41年,深圳大学秉承“自立、自律、自强”的校训,紧随特区,锐意改革、快速发展,为特区发展和国家现代化建设做出了重要贡献。");
25     }
26
27     // 运行 main | 调试 main | Run | Debug
28     public static void main(String[] args) {
29         // 创建一个JFrame对象, 设置标题为“中英互译 Demo”
30         // 创建一个名为“中英互译 Demo”的JFrame窗口
31         JFrame frame = new JFrame(title:"中英互译 Demo");
32         // 设置窗口关闭时的默认操作
33         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
34         // 设置窗口大小为700x400
35         frame.setSize(width:700, height:400);
36         // 设置窗口布局为3x3的网格布局
37         frame.setLayout(new GridLayout(rows:3, cols:3));
```

图五 2.1.A 程序内容①

```

38 // 创建一个标签, 显示中文句子
39 JLabel label1 = new JLabel(text:"中文句子:");
40 // 创建一个文本区域, 用于输入中文句子
41 JTextArea textArea1 = new JTextArea();
42 // 设置文本区域自动换行
43 textArea1.setLineWrap(wrap:true);
44 // 设置文本区域在换行时保持单词的完整性
45 textArea1.setWrapStyleWord(word:true);
46 // 设置文本区域自动换行
47 textArea1.setLineWrap(wrap:true);
48 // 设置文本区域在换行时保持单词的完整性
49 textArea1.setWrapStyleWord(word:true);
50 // 创建一个按钮, 按钮上显示“中译英”
51 JButton button1 = new JButton(text:"中译英");
52
53 // 创建一个标签, 标签上显示“英文句子:”
54 JLabel label2 = new JLabel(text:"英文句子:");
55 // 创建一个文本区域
56 JTextArea textArea2 = new JTextArea();
57 // 设置文本区域自动换行
58 textArea2.setLineWrap(wrap:true);
59 // 设置文本区域在换行时保持单词的完整性
60 textArea2.setWrapStyleWord(word:true);
61 // 创建一个按钮, 按钮上显示“英译中”
62 JButton button2 = new JButton(text:"英译中");
63 // 创建一个标签, 显示文本“这里是分界线~”
64 JLabel label3 = new JLabel(text:"这里是分界线~");
65 // 创建一个标签, 显示文本“请在上面填入中文或在下面填入英文”
66 JLabel label4 = new JLabel(text:"请在上面填入中文或在下面填入英文");
67 // 创建一个标签, 显示文本“请点击按钮进行翻译~”
68 JLabel label5 = new JLabel(text:"请点击按钮进行翻译~");
69
70 // 为按钮1添加监听器
71 button1.addActionListener(new ActionListener() {
72     // 重写actionPerformed方法
73     @Override
74     public void actionPerformed(ActionEvent e) {
75         // 获取文本框1中的文本
76         String chineseText = textArea1.getText();
77         // 获取翻译
78         String englishTranslation = translations.getOrDefault(chineseText, defaultValue:"翻译未找到");
79         // 将翻译设置到文本框2中
80         textArea2.setText(englishTranslation);
81     }
82 });
83
84 // 为按钮2添加监听器
85 button2.addActionListener(new ActionListener() {
86     // 重写actionPerformed方法
87     @Override
88     public void actionPerformed(ActionEvent e) {
89         // 获取文本框2中的英文文本
90         String englishText = textArea2.getText();
91         // 获取英文文本对应的中文翻译, 如果未找到则返回“翻译未找到”
92         String chineseTranslation = translations.getOrDefault(englishText, defaultValue:"翻译未找到");
93         // 将中文翻译设置到文本框1中
94         textArea1.setText(chineseTranslation);
95     }
96 });
97
98 // 将label1添加到frame中
99 frame.add(label1);
100 // 将textArea1添加到frame中
101 frame.add(textArea1);
102 // 将button1添加到frame中
103 frame.add(button1);
104 // 将label3添加到frame中
105 frame.add(label3);
106 // 将label4添加到frame中
107 frame.add(label4);
108 // 将label5添加到frame中
109 frame.add(label5);
110 // 将label2添加到frame中
111 frame.add(label2);
112 // 将textArea2添加到frame中
113 frame.add(textArea2);
114 // 将button2添加到frame中
115 frame.add(button2);
116
117 // 设置窗口可见
118 frame.setVisible(b:true);
119 }
120 }
121 }

```

图六 2.1.A 程序内容②

程序界面、结果：



图七 2.1.A 程序界面及例句翻译



图八 2.1.A 程序界面及非例句翻译

B、使用百度翻译 API、有道翻译 API 或其他 API 中的一个 API（此部分占 5 分）；
程序说明：

程序首先创建一个包含翻译对的静态映射 `translations`，然后定义了一个 `decodeUnicode` 方法用于解码 Unicode 字符串。在 `main` 方法中，程序创建了一个带有多个组件的 Swing GUI，包括标签、文本区域和按钮。按钮的点击事件会触发翻译操作，调用 `translateZhToEn` 或 `translateEnToZh` 方法对相应文本区域内提取的文字进行翻译，并将结果显示在相应的文本区域中。翻译方法通过调用百度翻译 API 实现，使用 MD5 生成签名，并处理 API 返回的 JSON 数据，然后将数据传入到相应的文本区域中。

程序内容：

```

1 package com.chino;
2
3 import java.awt.GridLayout;
4 import java.awt.event.ActionEvent;
5 import java.awt.event.ActionListener;
6 import java.io.BufferedReader;
7 import java.io.InputStreamReader;
8 import java.net.HttpURLConnection;
9 import java.net.URL;
10 import java.net.URLEncoder;
11 import java.security.MessageDigest;
12 import java.util.HashMap;
13 import java.util.Map;
14 import java.util.Random;
15
16 import javax.swing.JButton;
17 import javax.swing.JFrame;
18 import javax.swing.JLabel;
19 import javax.swing.JTextArea;
20
21 public class Chino_1 {
22     public static final Map<String, String> translations = new HashMap<>();
23
24     static {
25         translations.put(key:"建校41年[ ]深圳大学秉承“自立、自律、自强”的校训[ ]紧随特区[ ]锐意改革、快速发展[ ]为特区发展和国家现
26             value:"Sticking to the motto of "self-reliance, self-discipline, self-improvement", the University is
27         translations.put(
28             key:"Sticking to the motto of "self-reliance, self-discipline, self-improvement", the University is de
29             value:"建校41年[ ]深圳大学秉承“自立、自律、自强”的校训[ ]紧随特区[ ]锐意改革、快速发展[ ]为特区发展和国家现代化建设
30     }
31
32     public static String decodeUnicode(String unicodeStr) {
33         StringBuilder sb = new StringBuilder();
34         String[] hex = unicodeStr.split(regex:"\\\\\\u");
35         for (int i = 1; i < hex.length; i++) {
36             int data = Integer.parseInt(hex[i], radix:16);
37             sb.append((char) data);
38         }
39         return sb.toString();
40     }
41
42     运行 main | 调试 main | Run | Debug
43     public static void main(String[] args) {
44         JFrame frame = new JFrame(title:"中英互译 Demo");
45         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
46         frame.setSize(width:700, height:400);
47         frame.setLayout(new GridLayout(rows:3, cols:3));
48
49         JLabel label1 = new JLabel(text:"中文句子:");
50         JTextArea textArea1 = new JTextArea();
51         textArea1.setLineWrap(wrap:true);
52         textArea1.setStyleWord(word:true);
53         JButton button1 = new JButton(text:"中译英");
54
55         JLabel label2 = new JLabel(text:"英文句子:");
56         JTextArea textArea2 = new JTextArea();
57         textArea2.setLineWrap(wrap:true);
58         textArea2.setStyleWord(word:true);
59         JButton button2 = new JButton(text:"英译中");
60         JLabel label3 = new JLabel(text:"这里是分界线~");
61         JLabel label4 = new JLabel(text:"请在上面填入中文或在下面填入英文");
62         JLabel label5 = new JLabel(text:"请点击按钮进行翻译~");
63         button1.addActionListener(new ActionListener() {
64             @Override
65             public void actionPerformed(ActionEvent e) {
66                 String chineseText = textArea1.getText();
67                 String englishTranslation = translations.getDefault(chineseText, defaultValue:"翻译未找到");
68                 if ("翻译未找到".equals(englishTranslation)) {
69                     englishTranslation = translateZhToEn(chineseText);
70                 }
71                 textArea2.setText(englishTranslation);
72             }
73         });
74         button2.addActionListener(new ActionListener() {
75             @Override
76             public void actionPerformed(ActionEvent e) {
77                 String englishText = textArea2.getText();
78                 String chineseTranslation = translations.getDefault(englishText, defaultValue:"翻译未找到");
79                 if ("翻译未找到".equals(chineseTranslation)) {
80                     chineseTranslation = translateEnToZh(englishText);
81                 }
82                 textArea1.setText(chineseTranslation);
83             }
84         });
85
86         frame.add(label1);
87         frame.add(textArea1);
88         frame.add(button1);
89         frame.add(label3);
90         frame.add(label4);
91         frame.add(label5);
92         frame.add(label2);
93         frame.add(textArea2);
94         frame.add(button2);
95
96         frame.setVisible(b:true);
97     }
98

```

图九 2.1.B 程序内容①

```

99 private static String translateZhToEn(String query) {
100     String appId = "20241206002221426";
101     String secretKey = "i4nsPEZq9ISZ09KuvJEy";
102     String salt = String.valueOf(new Random().nextInt(bound:10000));
103     String sign = md5(appId + query + salt + secretKey);
104     String url = "http://api.fanyi.baidu.com/api/trans/vip/translate?q=" + encode(query) + "&from=zh&to=en&appid="
105         + appId + "&salt=" + salt + "&sign=" + sign;
106
107     try {
108         HttpURLConnection conn = (HttpURLConnection) new URL(url).openConnection();
109         conn.setRequestMethod(method:"GET");
110         BufferedReader in = new BufferedReader(new InputStreamReader(conn.getInputStream()));
111         StringBuilder result = new StringBuilder();
112         String line;
113         while ((line = in.readLine()) != null) {
114             result.append(line);
115         }
116         in.close();
117         // 解析返回的JSON数据
118         String response = result.toString();
119         System.err.println(response);
120         // 获取dst字段的起始位置
121         int start = response.indexOf(str:"\\\"dst\\\":\\\"") + 7;
122         // 获取dst字段的结束位置
123         int end = response.indexOf(str:"\\\"", start);
124         // 获取dst字段的值并进行URL解码
125         String translatedText = response.substring(start, end);
126         System.err.println("translatedText:" + translatedText);
127         if((translatedText.charAt(index:0)=='\\')&&(translatedText.charAt(index:1)=='u')){
128             translatedText = decodeUnicode(translatedText);
129         }
130         else if(translatedText.equals(anObject:"r_code")){
131             String temp="";
132             return temp;
133         }
134         System.err.println("translatedText:" + translatedText);
135         return translatedText;
136     } catch (Exception e) {
137         e.printStackTrace();
138         return "翻译失败";
139     }
140 }
141
142 private static String translateEnToZh(String query) {
143     String appId = "20241206002221426"; // 替换为你的百度翻译API的App ID
144     String secretKey = "i4nsPEZq9ISZ09KuvJEy"; // 替换为你的百度翻译API的Secret Key
145     String salt = String.valueOf(new Random().nextInt(bound:10000));
146     String sign = md5(appId + query + salt + secretKey);
147     String url = "http://api.fanyi.baidu.com/api/trans/vip/translate?q=" + encode(query) + "&from=en&to=zh&appid="
148         + appId + "&salt=" + salt + "&sign=" + sign;
149
150     try {
151         HttpURLConnection conn = (HttpURLConnection) new URL(url).openConnection();
152         conn.setRequestMethod(method:"GET");
153         BufferedReader in = new BufferedReader(new InputStreamReader(conn.getInputStream()));
154         StringBuilder result = new StringBuilder();
155         String line;
156         while ((line = in.readLine()) != null) {
157             result.append(line);
158         }
159         in.close();
160         // 解析返回的JSON数据
161         String response = result.toString();
162         System.err.println(response);
163         // 获取dst字段的起始位置
164         int start = response.indexOf(str:"\\\"dst\\\":\\\"") + 7;
165         // 获取dst字段的结束位置
166         int end = response.indexOf(str:"\\\"", start);
167         // 获取dst字段的值并进行URL解码
168         String translatedText = response.substring(start, end);
169         //translatedText = decodeUnicode(translatedText);
170         System.err.println("translatedText: " + translatedText);
171         if(translatedText.equals(anObject:"r_code")){
172             String temp="";
173             return temp;
174         }
175     }
176     // 遍历翻译后的文本
177     for(int i=0;i<translatedText.length();i++){
178         // 定义临时字符串
179         String temp="";
180         // 如果翻译后的文本中存在反斜杠
181         if(translatedText.charAt(i)=='\\'){
182             // 获取反斜杠后的三个字符
183             temp=translatedText.substring(i,i+6);
184             // 将这三个字符解码
185             String temp2=decodeUnicode(temp);
186             // 将解码后的字符串替换原来的字符串
187             translatedText=translatedText.replace(temp,temp2);
188         }
189     }
190     System.err.println("translatedText: " + translatedText);
191     return translatedText;
192 } catch (Exception e) {
193     e.printStackTrace();
194     return "翻译失败";
195 }

```

图十 2.1.B 程序内容②

```

197     private static String md5(String input) {
198         try {
199             MessageDigest md = MessageDigest.getInstance("MD5");
200             byte[] array = md.digest(input.getBytes());
201             StringBuilder sb = new StringBuilder();
202             for (byte b : array) {
203                 sb.append(String.format(format:"%02x", b));
204             }
205             return sb.toString();
206         } catch (Exception e) {
207             e.printStackTrace();
208             return null;
209         }
210     }
211
212     private static String encode(String input) {
213         try {
214             return URLEncoder.encode(input, enc:"UTF-8");
215         } catch (Exception e) {
216             e.printStackTrace();
217             return null;
218         }
219     }
220 }

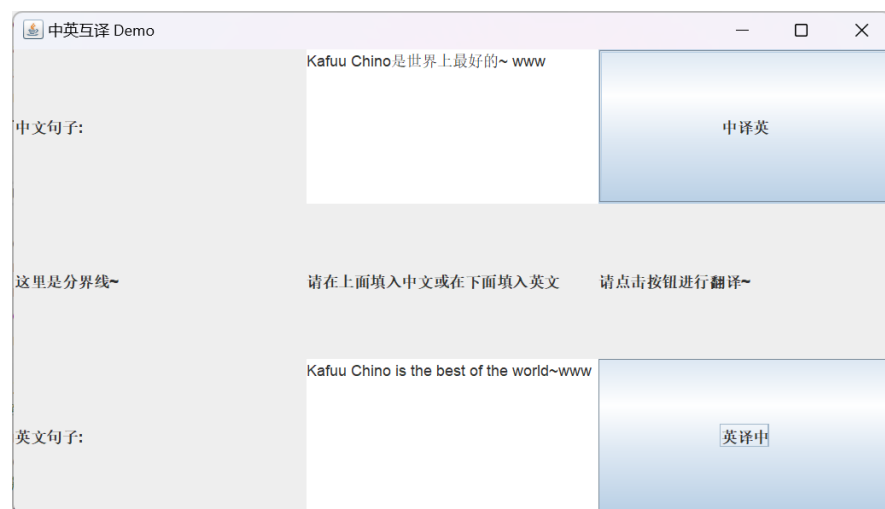
```

图十一 2.1.B 程序内容③

程序界面、结果：



图十二 2.1.B 程序界面及中译英结果



图十三 2.1.B 程序界面及英译中结果

C、使用腾讯混元大模型、百度文心一言大模型 API、阿里通义千问大模型 API 或其他大模型 API 中的两个 API（此部分占 10 分）；

阿里通义千问大模型 API：

程序说明：

程序首先创建一个 JFrame 对象，并设置其标题、关闭操作、大小和布局。接着，程序创建多个 Swing 组件，包括标签、文本区域和按钮，并将它们添加到 JFrame 中。按钮的点击事件会触发对话操作，读取相应文本区域内的文本后调用 callWithMessage 方法进行对话，并将结果显示在相应的文本区域中。callWithMessage 方法通过调用通义千问 API 实现，使用 API Key 和对话内容生成请求，并处理 API 返回的结果后将结果传入相应的文本区域之中。

程序内容：

```
1 package com.chino;
2
3 import java.awt.GridLayout;
4 import java.awt.event.ActionEvent;
5 import java.awt.event.ActionListener;
6 import java.util.Arrays;
7
8 import javax.swing.JButton;
9 import javax.swing.JFrame;
10 import javax.swing.JLabel;
11 import javax.swing.JTextArea;
12
13 import com.alibaba.dashscope.aigc.generation.Generation;
14 import com.alibaba.dashscope.aigc.generation.GenerationParam;
15 import com.alibaba.dashscope.aigc.generation.GenerationResult;
16 import com.alibaba.dashscope.common.Message;
17 import com.alibaba.dashscope.common.Role;
18 import com.alibaba.dashscope.exception.ApiException;
19 import com.alibaba.dashscope.exception.InputRequiredException;
20 import com.alibaba.dashscope.exception.NoApiKeyException;
21
22 public class Chino_3 {
23     Run | Debug | 运行 main | 测试 main.
24     public static void main(String[] args) {
25         // 创建一个 JFrame 对象，设置标题为“通义千问对话 Demo”
26         JFrame frame = new JFrame(title:"通义千问对话 Demo");
27         // 设置关闭窗口时退出程序
28         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
29         // 设置窗口大小为 700x400
30         frame.setSize(width:700, height:400);
31         // 设置窗口布局为 3x3 的网格布局
32         frame.setLayout(new GridLayout(rows:3, cols:3));
33
34         // 创建一个标签，显示文本“对话输入框~”
35         JLabel label1 = new JLabel(text:"对话输入框~");
36         // 创建一个文本区域，用于输入对话内容
37         JTextArea textArea1 = new JTextArea();
38         // 设置文本区域自动换行
39         textArea1.setLineWrap(wrap:true);
40         // 设置文本区域在换行时保持单词的完整性
41         textArea1.setWrapStyleWord(word:true);
42         JButton button1 = new JButton(text:"对话");
43
44         // 创建一个标签，显示文本“AI回答:”
45         JLabel label2 = new JLabel(text:"AI回答:");
46         // 创建一个文本区域，用于显示AI的回答
47         JTextArea textArea2 = new JTextArea();
48         // 设置文本区域自动换行
49         textArea2.setLineWrap(wrap:true);
50         // 设置文本区域在换行时保持单词的完整性
51         textArea2.setWrapStyleWord(word:true);
52         // 创建一个按钮，用于进行对话
53         JButton button2 = new JButton(text:"");
54         // 创建一个标签，显示文本“这里是分界线~”
55         JLabel label3 = new JLabel(text:"这里是分界线~");
56         // 创建一个标签，显示文本“这里是分界线~”
57         JLabel label4 = new JLabel(text:"这里是分界线~");
58         // 创建一个标签，显示文本“请点击按钮进行对话~”
59         JLabel label5 = new JLabel(text:"请点击按钮进行对话~");
60         // 为按钮1添加监听器
61         button1.addActionListener(new ActionListener() {
62             @Override
63             public void actionPerformed(ActionEvent e) {
64                 String chineseText = textArea1.getText();
65                 String englishTranslation = "";
66                 try {
67                     englishTranslation = callWithMessage(chineseText, from:"zh", to:"en");
68                 } catch (Exception ex) {
69                     ex.printStackTrace();
70                     englishTranslation = "对话失败,请更换对话内容";
71                 }
72                 textArea2.setText(englishTranslation);
73             }
74         });
75     }
76 }
```

图十四 2.1.C 阿里通义千问大模型 AI 对话程序内容①

```

75     frame.add(label1);
76     // 添加标签1到框架中
77     frame.add(textArea1);
78     // 添加文本区域1到框架中
79     frame.add(button1);
80     // 添加按钮1到框架中
81     frame.add(label3);
82     // 添加标签3到框架中
83     frame.add(label4);
84     // 添加标签4到框架中
85     frame.add(label5);
86     // 添加标签5到框架中
87     frame.add(label2);
88     // 添加标签2到框架中
89     frame.add(textArea2);
90     // 添加文本区域2到框架中
91     //frame.add(button2);
92
93     frame.setVisible(b:true);
94 }
95 //调用通义千问API进行对话
96 //以下大部分为通义千问提供的模板
97 private static String callWithMessage(String content, String from, String to) throws ApiException, NoApiKeyException, InputRequiredException {
98     Generation gen = new Generation();
99     Message systemMsg = Message.builder()
100         .role(Role.SYSTEM.getValue())
101         .content(content:"You are a helpful assistant.")
102         .build();
103     Message userMsg = Message.builder()
104         .role(Role.USER.getValue())
105         .content(content)//传入对话内容
106         .build();
107     GenerationParam param = GenerationParam.builder()
108         .apiKey(System.getenv(name:"DASHSCOPE_API_KEY"))//传入API_KEY
109         .model(model:"qwen-plus")
110         .messages(Arrays.asList(systemMsg, userMsg))
111         .resultFormat(GenerationParam.ResultFormat.MESSAGE)
112         .build();
113     GenerationResult result = gen.call(param);
114     return result.getOutput().getChoices().get(index:0).getMessage().getContent();//返回回答内容
115 }
116 }

```

图十五 2.1.C 阿里通义千问大模型 AI 对话程序内容②

程序界面、结果：



图十六 2.1.C 阿里通义千问大模型 API 对话程序界面及结果

腾讯混元大模型：

程序说明：

程序首先创建一个 JFrame 窗口，并设置其标题、关闭操作、大小和布局。接着，程序创建多个 Swing 组件，包括标签、文本区域和按钮，并将它们添加到 JFrame 中。按钮的点击事件会触发对话操作，读取相应文本区域内的文本后，调用 callTencentCloudAPI 方法进行对话，并将结果显示在相应的文本区域中。callTencentCloudAPI 方法通过调用腾讯混元大模型 API 实现，使用凭证和请求参数生成请求，并处理 API 返回的结果。最后，设置窗口可见。

程序内容:

```
1 package com.chino;
2
3 import java.awt.*;
4 import java.awt.event.ActionEvent;
5 import java.awt.event.ActionListener;
6 import javax.swing.*;
7
8 import com.tencentcloudapi.common.SSEResponseModel;
9 import com.tencentcloudapi.common.AbstractModel;
10 import com.tencentcloudapi.common.Credential;
11 import com.tencentcloudapi.common.profile.ClientProfile;
12 import com.tencentcloudapi.common.profile.HttpProfile;
13 import com.tencentcloudapi.common.exception.TencentCloudSDKException;
14 import com.tencentcloudapi.hunyuan.v20230901.HunyuanClient;
15 import com.tencentcloudapi.hunyuan.v20230901.models.*;
16
17 public class Chino_5 {
18     // Run [Debug] 运行 main 测试 main
19     public static void main(String[] args) {
20         // 创建一个JFrame窗口, 设置窗口标题为"腾讯混元大模型对话 Demo"
21         JFrame frame = new JFrame(title:"腾讯混元大模型对话 Demo");
22         // 设置窗口关闭时的默认操作为退出程序
23         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
24         // 设置窗口大小为700x400
25         frame.setSize(width:700, height:400);
26         // 设置窗口布局为3行3列的网格布局
27         frame.setLayout(new GridLayout(rows:3, cols:3));
28
29         // 创建一个JLabel标签, 显示文本"对话输入框~"
30         JLabel label1 = new JLabel(text:"对话输入框~");
31         // 创建一个JTextArea文本域, 用于显示对话内容
32         JTextArea textArea1 = new JTextArea();
33         // 设置文本域自动换行
34         textArea1.setLineWrap(wrap:true);
35         // 设置文本域在换行时保持单词的完整性
36         textArea1.setWrapStyleWord(word:true);
37         // 创建一个JButton按钮, 显示文本"对话~"
38         JButton button1 = new JButton(text:"对话~");
39
40         // 创建一个标签, 显示文本"AI回答:"
41         JLabel label2 = new JLabel(text:"AI回答:");
42         // 创建一个文本区域, 用于显示AI的回答
43         JTextArea textArea2 = new JTextArea();
44         // 设置文本区域自动换行
45         textArea2.setLineWrap(wrap:true);
46         // 设置文本区域在换行时保持单词的完整性
47         textArea2.setWrapStyleWord(word:true);
48         // 创建一个按钮, 用于触发对话
49         JButton button2 = new JButton(text:"");
50         // 创建一个标签, 显示文本"这里是分界线~"
51         JLabel label3 = new JLabel(text:"这里是分界线~");
52         // 创建一个标签, 显示文本"这里是分界线~"
53         JLabel label4 = new JLabel(text:"这里是分界线~");
54         // 创建一个标签, 显示文本"请点击按钮进行对话~"
55         JLabel label5 = new JLabel(text:"请点击按钮进行对话~");
56         // 为按钮添加一个监听器, 当按钮被点击时触发事件
57         // 为按钮1添加监听器
58         button1.addActionListener(new ActionListener() {
59             // 重写ActionPerformed方法
60             @Override
61             public void actionPerformed(ActionEvent e) {
62                 // 获取文本框1中的文本
63                 String inputText = textArea1.getText();
64                 // 调用腾讯云API, 获得输出文本
65                 String responseText = callTencentCloudAPI(inputText);
66                 // 将输出文本设置到文本框2中
67                 textArea2.setText(responseText);
68             }
69         });
70         // 将标签、文本域和按钮添加到窗口中
71         frame.add(label1);
72         frame.add(textArea1);
73         frame.add(button1);
74         frame.add(label3);
75         frame.add(label4);
76         frame.add(label5);
77         frame.add(textArea2);
78         frame.add(button2);
79
80         // 设置窗口可见
81         frame.setVisible(true);
82     }
83
84     // 调用腾讯混元大模型API进行对话
85     // 以下大部分为腾讯云提供的模板
86     private static String callTencentCloudAPI(String content) {
87         // 传入所需内容
88         try {
89             Credential cred = new Credential(secretId:"AKIDb8psvuhHtUqANNE4kHbu40IXGwS5lN", secretKey:"MzfquzxmX9Yix2askgjeejx8PYZD");
90             HttpProfile httpProfile = new HttpProfile();
91             httpProfile.setEndpoint(endpoint:"hunyuan.tencentcloudapi.com");
92             ClientProfile clientProfile = new ClientProfile();
93             clientProfile.setHttpProfile(httpProfile);
94             HunyuanClient client = new HunyuanClient(cred, "ap-guangzhou", clientProfile);
95             ChatCompletionsRequest req = new ChatCompletionsRequest();
96             req.setModel("hunyuan-lite");
97             Message message = new Message();
98             message.setRole("user");
99             message.setContent(content);
100             req.setMessages(new Message[]{message});
101             ChatCompletionsResponse resp = client.ChatCompletions(req);
102             String responseContent;
103             if (resp.isStream()) {
104                 StringBuilder result = new StringBuilder();
105                 for (SSEResponseModel.SSE e : resp) {
106                     result.append(e.Data);
107                 }
108                 responseContent = result.toString();
109             } else {
110                 responseContent = AbstractModel.toJsonString(resp);
111             }
112             // 提取"Content"字段后的文字
113             int start = responseContent.indexOf(("\\Content\\:")) + 11;
114             int end = responseContent.indexOf("\\", start);
115             String extractedContent = responseContent.substring(start, end);
116             // 替换\n为正常的换行符, 并去除其他符号
117             extractedContent = extractedContent.replace(target:"\\n", replacement:"\\n").replaceAll(regex:"\\\\\\\\\\\\", replacement:"");
118             return extractedContent;
119         } catch (TencentCloudSDKException e) {
120             //System.err.println(e.toString());
121             return e.toString();
122         }
123     }
124 }
125 }
```

图十七 2.1.C 腾讯混元大模型 API 对话程序内容

程序界面、结果：



图十八 2.1.C 腾讯混元大模型 API 对话程序界面及结果

Part 3 (30 分)

(3.1). 利用套接字连接 (TCP) 编写程序, 该程序包括三个客户端 (ClientA、ClientB、ClientC) 和一个服务端 (ServerS), 三个客户端通过服务端作为桥梁实现相互间的文字交流, 例如, ClientA 先发信息给 ServerS, 然后 ServerC 再将收到的信息转发给 ClientB 和 ClientC。在报告中附上示意图 (三个客户端+一个服务端)、程序截图、完整的运行结果和简要文字说明。(20 分)

Client. java:

程序说明:

程序首先创建一个聊天客户端的 GUI 界面, 并初始化输入输出流。接着, 程序连接到本地服务器, 并通过套接字与服务器进行通信。用户在文本框中输入消息后, 消息会发送到服务器, 同时客户端会不断接收并显示来自服务器的消息。最后, 程序在关闭时会隐藏并销毁窗口

程序内容:

```
1 package com.chino;
2
3 import java.awt.BorderLayout;
4 import java.awt.event.ActionEvent;
5 import java.awt.event.ActionListener;
6 import java.io.BufferedReader;
7 import java.io.IOException;
8 import java.io.InputStreamReader;
9 import java.io.PrintWriter;
10 import java.net.Socket;
11
12 import javax.swing.JFrame;
13 import javax.swing.JScrollPane;
14 import javax.swing.JTextArea;
15 import javax.swing.JTextField;
16
17 public class Client {
18     // 定义一个BufferedReader对象, 用于读取输入
19     private BufferedReader in;
20     // 定义一个PrintWriter对象, 用于输出
21     private PrintWriter out;
22     // 定义一个String对象, 用于存储用户名
23     private String name;
24     // 定义一个JFrame对象, 用于创建聊天客户端窗口
25     private JFrame frame = new JFrame(title:"聊天客户端");
26     // 定义一个JTextField对象, 用于输入消息
27     private JTextField textField = new JTextField(columns:40);
28     // 定义一个JTextArea对象, 用于显示消息
29     private JTextArea messageArea = new JTextArea(rows:8, columns:40);
```

图十九 3.1 Client.java 程序内容①

```

31     public Client(String name) {
32         // 构造函数，初始化客户端
33         this.name = name;
34
35         // 设置文本框可编辑
36         textField.setEditable(b:true);
37         // 设置消息区域不可编辑
38         messageArea.setEditable(b:false);
39         // 将文本框添加到窗口的南部
40         frame.getContentPane().add(textField, BorderLayout.SOUTH);
41         // 将消息区域添加到窗口的中央，并添加滚动条
42         frame.getContentPane().add(new JScrollPane(messageArea), BorderLayout.CENTER);
43
44         // 为文本框添加监听器，当按下回车键时，将文本框中的内容发送出去，并清空文本框
45         textField.addActionListener(new ActionListener() {
46             public void actionPerformed(ActionEvent e) {
47                 out.println(textField.getText());
48                 textField.setText("");
49             }
50         });
51     }
52
53     // 运行方法，抛出IOException异常
54     private void run() throws IOException {
55         try {
56             // 创建Socket对象，连接到本地主机的12345端口
57             Socket socket = new Socket(host:"localhost", port:12345);
58             // 创建BufferedReader对象，读取Socket的输入流
59             in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
60             // 创建PrintWriter对象，写入Socket的输出流
61             out = new PrintWriter(socket.getOutputStream(), autoFlush:true);
62
63             // 向服务器发送用户名
64             out.println(name);
65
66             // 无限循环，读取服务器发送的消息
67             while (true) {
68                 // 读取一行消息
69                 String line = in.readLine();
70                 // 如果消息为空，则退出循环
71                 if (line == null) {
72                     return;
73                 }
74                 // 将消息添加到消息区域
75                 messageArea.append(line + "\n");
76             }
77         } finally {
78             // 关闭窗口
79             frame.setVisible(b:false);
80             frame.dispose();
81         }
82     }
83
84     // 创建并显示GUI，传入客户端名称
85     public static void createAndShowGUI(final String clientName) {
86         // 创建客户端对象，传入客户端名称
87         Client client = new Client(clientName);
88         // 设置窗口关闭时的默认操作
89         client.frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
90         // 调整窗口大小以适应其子组件的首选大小
91         client.frame.pack();
92         // 设置窗口可见
93         client.frame.setVisible(b:true);
94         // 运行客户端
95         try {
96             client.run();
97         } catch (IOException e) {
98             // 打印异常堆栈信息
99             e.printStackTrace();
100         }
101     }
102 }

```

图二十 3.1 Client.java 程序内容②

ClientA.java、ClientB.java、ClientC.java:

程序说明:

程序首先调用 Client.createAndShowGUI 方法创建并显示名为“ClientA”、“ClientB”、“ClientC”的聊天客户端 GUI 界面。接着,程序通过 main 方法启动客户端。

程序内容:

```
1  package com.chino;
2
3  public class ClientA{
4      |   Run | Debug | 运行 main | 调试 main,
      |   public static void main(String[] args) {
5      |       |   Client.createAndShowGUI(clientName:"ClientA");
6      |       |   }
7      |   }

1  package com.chino;
2
3  public class ClientB {
4      |   Run | Debug | 运行 main | 调试 main,
      |   public static void main(String[] args) {
5      |       |   Client.createAndShowGUI(clientName:"ClientB");
6      |       |   }
7      |   }

1  package com.chino;
2
3  public class ClientC {
4      |   Run | Debug | 运行 main | 调试 main,
      |   public static void main(String[] args) {
5      |       |   Client.createAndShowGUI(clientName:"ClientC");
6      |       |   }
7      |   }
```

图二十一到图二十四 3.1 ClientA.java、ClientB.java、ClientC.java 程序内容

ServerS.java:

程序说明:

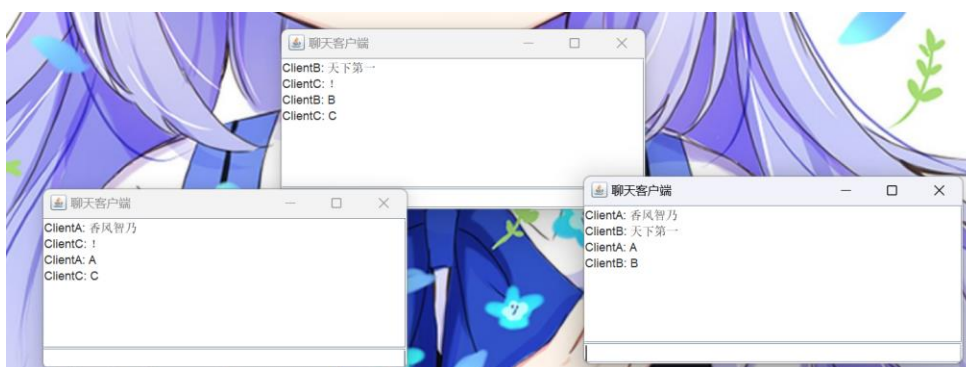
程序首先启动一个聊天服务器,并监听端口 12345。接着,程序接受客户端连接,并为每个连接创建一个处理线程。每个线程读取客户端的消息,并将消息广播给所有其他已连接的客户端。最后,程序在客户端断开连接时移除相应的输出流。

程序内容:

```
1 package com.chino;
2
3 import java.io.BufferedReader;
4 import java.io.IOException;
5 import java.io.InputStreamReader;
6 import java.io.PrintWriter;
7 import java.net.ServerSocket;
8 import java.net.Socket;
9 import java.util.HashSet;
10
11 public class ServerS {
12     private static final int PORT = 12345; // 定义服务器端口号
13     private static HashSet<PrintWriter> writers = new HashSet<>(); // 定义一个HashSet, 用于存储PrintWriter对象
14
15     Run | Debug | 运行 main | 调试 main.
16     public static void main(String[] args) throws Exception {
17         System.out.println(x:"聊天服务器已启动..."); // 输出服务器已启动的信息
18         ServerSocket listener = new ServerSocket(PORT); // 创建ServerSocket对象, 监听指定端口
19         try {
20             while (true) {
21                 new Handler(listener.accept()).start(); // 接受客户端连接, 并创建一个新的Handler对象处理连接
22             }
23         } finally {
24             listener.close(); // 关闭ServerSocket对象
25         }
26     }
27
28     private static class Handler extends Thread {
29         private Socket socket; // 声明一个Socket对象
30         private PrintWriter out; // 声明一个PrintWriter对象
31         private BufferedReader in; // 声明一个BufferedReader对象
32         private String name; // 声明一个String类型的name变量
33
34         public Handler(Socket socket) { // 构造函数, 传入一个Socket对象
35             this.socket = socket; // 将传入的Socket对象赋值给成员变量socket
36         }
37
38         public void run() {
39             try {
40                 // 创建输入流
41                 in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
42                 // 创建输出流
43                 out = new PrintWriter(socket.getOutputStream(), autoFlush:true);
44
45                 // 读取客户端发送的名称
46                 name = in.readLine();
47                 // 输出客户端名称
48                 System.out.println(name + " 已连接");
49                 // 同步代码块, 保证线程安全
50                 synchronized (writers) {
51                     // 将输出流添加到writers集合中
52                     writers.add(out);
53                 }
54
55                 String input;
56                 while ((input = in.readLine()) != null) { // 读取一行输入, 直到输入为空
57                     for (PrintWriter writer : writers) { // 遍历所有输出流
58                         if (writer != out) { // 不发送给发送者自己
59                             writer.println(name + ": " + input);
60                         }
61                     }
62                 }
63             } catch (IOException e) {
64                 // 捕获IO异常并打印异常信息
65                 System.out.println(e);
66             } finally {
67                 // 如果输出流不为空, 则从writers中移除
68                 if (out != null) {
69                     writers.remove(out);
70                 }
71                 // 关闭socket连接
72                 try {
73                     socket.close();
74                 } catch (IOException e) {}
75             }
76         }
77     }
78 }
```

图二十五 3.1 ServerS.java 程序内容

程序界面、结果：



图二十六 3.1 程序结果

(3.2). 利用数据报通信（UDP）实现题(1)中的要求。（10分）

Client.java:

程序说明：

程序首先创建一个 UDP 聊天客户端的 GUI，包括文本输入框和消息显示区域。接着，程序初始化一个 DatagramSocket，并发送连接消息到服务器。用户输入消息后，程序会将消息通过 DatagramPacket 发送到服务器。同时，程序启动一个线程持续接收来自服务器的消息，并将其显示在消息区域中。最后，程序在 main 方法中创建客户端实例并运行。

程序内容：

```
1 package com.chino;
2
3 import java.awt.BorderLayout;
4 import java.io.IOException;
5 import java.net.DatagramPacket;
6 import java.net.DatagramSocket;
7 import java.net.InetAddress;
8
9 import javax.swing.JFrame;
10 import javax.swing.JScrollPane;
11 import javax.swing.JTextArea;
12 import javax.swing.JTextField;
13
14 public class Client {
15     private static final String SERVER_IP = "localhost";
16     private static final int SERVER_PORT = 12345;
17     private DatagramSocket socket;
18     private String name;
19     private JFrame frame;
20     private JTextField textField;
21     private JTextArea messageArea;
22
23     public Client(String name) {
24         this.name = name;
25
26         // 创建GUI
27         frame = new JFrame("UDP聊天客户端 - " + name);
28         textField = new JTextField(columns:40);
29         messageArea = new JTextArea(rows:8, columns:40);
30
31         textField.setEditable(true);
32         messageArea.setEditable(false);
33         frame.getContentPane().add(textField, BorderLayout.SOUTH);
34         frame.getContentPane().add(new JScrollPane(messageArea), BorderLayout.CENTER);
35
36         textField.addActionListener(e -> sendMessage(textField.getText()));
37
38         frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
39         frame.pack();
40         frame.setVisible(true);
41     }
42
43     private void sendMessage(String message) {
44         try {
45             byte[] sendData = message.getBytes();
46             DatagramPacket sendPacket = new DatagramPacket(
47                 sendData,
48                 sendData.length,
49                 InetAddress.getByAddress(SERVER_IP),
50                 SERVER_PORT
51             );
52             socket.send(sendPacket);
53             textField.setText("");
54         } catch (IOException e) {
55             e.printStackTrace();
56         }
57     }
58 }
```

图二十七 3.2 Client.java 程序内容①

```

59     private void run() throws IOException {
60         socket = new DatagramSocket();
61
62         // 发送连接消息
63         String connectMessage = "CONNECT:" + name;
64         byte[] connectData = connectMessage.getBytes();
65         DatagramPacket connectPacket = new DatagramPacket(
66             connectData,
67             connectData.length,
68             InetAddress.getByName(SERVER_IP),
69             SERVER_PORT
70         );
71         socket.send(connectPacket);
72
73         // 启动接收线程
74         new Thread(() -> {
75             try {
76                 byte[] receiveData = new byte[1024];
77                 while (true) {
78                     DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
79                     socket.receive(receivePacket);
80                     String message = new String(receivePacket.getData(), 0, receivePacket.getLength());
81                     messageArea.append(message + "\n");
82                 }
83             } catch (IOException e) {
84                 e.printStackTrace();
85             }
86         }).start();
87     }
88
89     Run | Debug | 运行 main | 调试 main,
90     public static void main(String[] args) {
91         String clientName = args[0];
92         Client client = new Client(clientName);
93         try {
94             client.run();
95         } catch (IOException e) {
96             e.printStackTrace();
97         }
98     }

```

图二十八 3.2 Client.java 程序内容②

ClientA.java、ClientB.java、ClientC.java:

程序说明:

程序在 main 方法中调用 Client 类的 main 方法,并传递参数"ClientA"、"ClientB"、"ClientC",以分别启动三个名字 UDP 聊天客户端。

程序内容:

```

1     package com.chino;
2
3     public class ClientB {
4         Run | Debug | 运行 main | 调试 main,
5         public static void main(String[] args) {
6             Client.main(new String[]{"ClientB"});
7         }
8     }
9
1    package com.chino;
2
3    public class ClientC {
4        Run | Debug | 运行 main | 调试 main,
5        public static void main(String[] args) {
6            Client.main(new String[]{"ClientC"});
7        }
8    }
9
1    package com.chino;
2
3    public class ClientA {
4        Run | Debug | 运行 main | 调试 main,
5        public static void main(String[] args) {
6            Client.main(new String[]{"ClientA"});
7        }
8    }

```

图二十九到图三十一 3.2 ClientA.java、ClientB.java、ClientC.java 程序内容

ServerS.java:

程序说明:

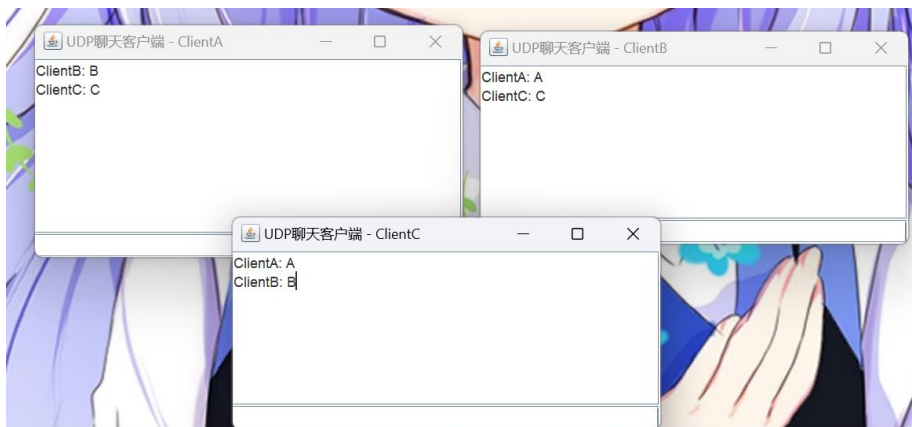
程序首先创建一个 UDP 服务器,并在指定端口上监听客户端连接。接着,程序使用一个 Set 来存储已连接的客户端信息。服务器持续接收客户端发送的数据包,并处理新客户端的连接请求,将其信息添加到客户端集合中。然后对于收到的消息,服务器会将其附加发送者的名称,转发给其他所有已连接的客户端。

程序内容:

```
1  package com.chino;
2
3  import java.io.IOException;
4  import java.net.DatagramPacket;
5  import java.net.DatagramSocket;
6  import java.net.InetAddress;
7  import java.util.HashSet;
8  import java.util.Set;
9
10 public class ServerS {
11     private static final int PORT = 12345;
12     private static Set<ClientInfo> clients = new HashSet<>();
13
14     static class ClientInfo {
15         InetAddress address;
16         int port;
17         String name;
18
19         public ClientInfo(InetAddress address, int port, String name) {
20             this.address = address;
21             this.port = port;
22             this.name = name;
23         }
24
25         @Override
26         public boolean equals(Object obj) {
27             if (obj instanceof ClientInfo) {
28                 ClientInfo other = (ClientInfo) obj;
29                 return address.equals(other.address) && port == other.port;
30             }
31             return false;
32         }
33
34         @Override
35         public int hashCode() {
36             return address.hashCode() + port;
37         }
38     }
39
40     Run | Debug | 运行 main | 测试 main
41     public static void main(String[] args) throws IOException {
42         DatagramSocket socket = new DatagramSocket(PORT);
43         System.out.println("UDP服务器已启动...");
44
45         byte[] receiveData = new byte[1024];
46
47         while (true) {
48             DatagramPacket receivePacket = new DatagramPacket(receiveData, receiveData.length);
49             socket.receive(receivePacket);
50
51             String message = new String(receivePacket.getData(), 0, receivePacket.getLength());
52             InetAddress clientAddress = receivePacket.getAddress();
53             int clientPort = receivePacket.getPort();
54
55             // 处理新客户端连接
56             if (message.startsWith(prefix:"CONNECT:")) {
57                 String clientName = message.substring(beginIndex:0);
58                 clients.add(new ClientInfo(clientAddress, clientPort, clientName));
59                 System.out.println(clientName + " 已连接");
60                 continue;
61             }
62
63             // 转发消息给其他客户端
64             for (ClientInfo client : clients) {
65                 if (client.port != clientPort || !client.address.equals(clientAddress)) {
66                     String senderName = "";
67                     for (ClientInfo c : clients) {
68                         if (c.port == clientPort && c.address.equals(clientAddress)) {
69                             senderName = c.name;
70                             break;
71                         }
72                     }
73
74                     String forwardMessage = senderName + ": " + message;
75                     byte[] sendData = forwardMessage.getBytes();
76                     DatagramPacket sendPacket = new DatagramPacket(
77                         sendData,
78                         sendData.length,
79                         client.address,
80                         client.port
81                     );
82                     socket.send(sendPacket);
83                 }
84             }
85         }
86     }
87 }
```

图三十二 3.2 ServerS.java 程序内容

程序界面、结果：



图三十三 3.2 程序结果

+++++

其他（例如感想、建议等等）。

在这次实验当中最难以攻克的就是大模型 AI 的部分了，在寻找的过程当中只找到阿里云和腾讯云有较为完备的 Java 引用例和操作流程，最后也理所当然的用了这两个进行 demo 的制作。

由于大模型 AI 的引用需要依赖大量其公司的自建库，由于 Vscode 本身的库管理系统是缺失的（默认的编译是随机配置库环境，也无法导入自建库），导致花了很多的时间在配置 Maven 上面，不过最后还是解决了自建库的配置问题，最终成功完成了 AI 大模型的实验。

在第三题的代码写作过程当中，发现由于 Vscode 的运行问题，几个端口无法同时展现在同一截图当中，就不得不在第二题当中的图形化过程上进一步对第三题的端口进行图形化，最后呈现出的便是实验报告中的效果。

深圳大学学生实验报告用纸

指导教师批阅意见：

成绩评定：

指导教师签字：

2024 年 月 日

备注：

注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。

2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。